

# File permissions in Linux

## Project description

In this project, I assume the role of a security professional in charge of auditing and managing file system permissions for a research team. The main objective is to ensure that all files and directories comply with the organization's authorization policies, applying the principle of least privilege to mitigate the risk of unauthorized access. I use Linux shell commands to examine existing permission strings and make the necessary modifications using the `chmod` utility.

## Check file and directory details

To start the audit, I navigated to the projects directory and executed the command `ls -la`. This command is fundamental because the `-l` option allows viewing the detailed permissions in a long list format, and the `-a` option ensures that hidden files (those that begin with a dot) are also visible.

According to the current file permissions document, the command output shows the following initial permissions:

project\_k.txt: -rw-rw-rw-

project\_m.txt: -rw-r-----

project\_r.txt: -rw-rw-r--

project\_t.txt: -rw-rw-r--

.project\_x.txt: -rw--w----

drafts (directory): drwx--x---

## Describe the permissions string

Taking the `project_r.txt` file as an example, its permission string is `-rw-rw-r--`. This 10-character string is interpreted as follows:

**First character (-):** Indicates the file type. The hyphen means it is a regular file (a `d` would indicate a directory).

**Characters 2-4 (rw-):** Represent the user's (owner's) permissions. In this case, they have read (r) and write (w) permissions, but not execute permissions.

**Characters 5-7 (rw-):** Represent the group's permissions. Like the owner, the group has read and write access.

**Characters 8-10 (r--):** Represent the permissions of other users. They only have read permission (r).

## Change file permissions

The organization's policy strictly prohibits "other users" from having write access to any file. Upon reviewing the list, I identified that `project_k.txt` violates this rule, as its original string is `-rw-rw-rw-`.

To correct this, I ran the following command: `chmod o-w project_k.txt`

**Explanation:** The `o-w` argument tells the system to remove the write permission (w) from the other users (o) category. After this change, the file had the permissions `-rw-r--`, aligning with the corporate security policy.

## Change file permissions on a hidden file

The hidden file `.project_x.txt` had a high-risk configuration: `-rw--w----`. The organization requires that no one have write permissions on this archived file, but that both the user and the group can read it.

To comply with this requirement, I used the command: `chmod u-w,g-w,g+r .project_x.txt`

**Explanation:** With this command, I removed write permission from the user (u-w) and the group (g-w), and added read permission for the group (g+r). The resulting new permission string is `-r--r-----`, ensuring the integrity of the archived data.

## Change directory permissions

The drafts directory initially had the permissions `drwx--x---`, which allowed the group to execute or enter the directory. However, the instruction indicates that only the user researcher2 should have access.

**To restrict access, I executed:** `chmod g-x drafts`

**Explanation:** By removing the execute permission for the group (g-x), any user other than the owner is prevented from accessing the contents of the folder. The final string is `drwx-----`.

## Summary

In this project, the permissions structure of the research directory was successfully audited and corrected. Through the use of `ls -la`, critical vulnerabilities were identified, such as excessive write permissions on public files and misconfigurations in hidden files and confidential directories. The application of `chmod` commands made it possible to restrict access in accordance with institutional policies, ensuring that only authorized users possess the strictly necessary privileges for their work, thereby strengthening the security posture of the system.